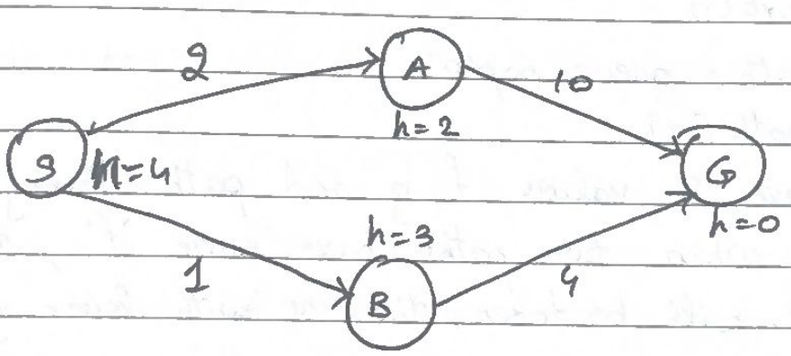
Lab-5 A* Search

To implement the A* search algorithm and find the path from start node to goal node using
- g(n) → actual cost from start node to n
- h(n) → estimated cost from n to goal
- f(n) → total estimated cost.

The formula is : $\boxed{f(n) = g(n) + h(n)}$

A* algorithm chooses the node with smallest f(n).



Step 1: Declaring the Graph & Heuristics

```
graph = {'S' :[['A', 2) , ('B', 1)] ,
         'A' : [('G', 10)] ,
         'B' : [['G', 4]] ,
         'G' : []}
h = {'S' : 4, 'A' : 2 , 'B' : 3, 'G' : 0}
```

Step 2: Check for overestimated Heuristics

```
true_cost = {'S': 5 , 'A' : 10 , 'B' : 4, 'G' :0}
for node in h:
    ok = "Ok" if h[node] <= true_cost[node] else "VIOLATION"
    print (f" h({node}) = {h[node]} true = {true_cost[node]} {ok}")
```

true_cost contains the actual cost from the every node to the goal.

we want $h(n) \leq true_cost(n)$

Step 3: Declare Function
def astar(start, goal)

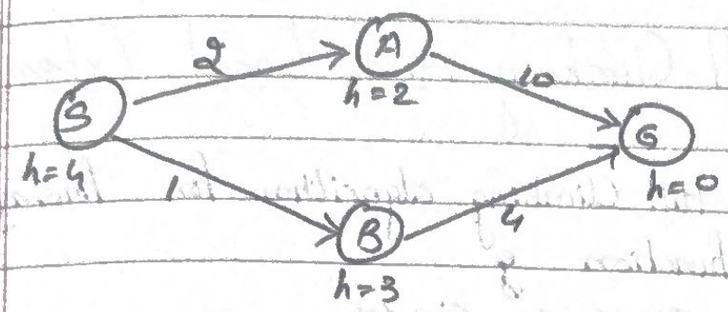
Step 4: Initialise the Queue
queue = [(h[start], 0, [start])]
The queue holds 3 values i.e queue (f, g, path)
Initially we are at S h(s) = 4, g(s) = 0 path [S]
therefore it contains [(4, 0, ['s'])]

Step 5: Unpack all 3
while queue:
    queue.sort()
    f, g, path = queue.pop(0)
    node = path[-1]
A* needs all 3 values f, g and path. During a tie in f(n) i.e when two paths have same 'f', the second element 'g' will be taken, the one with lesser g value is considered

Step 6: Goal Test
if node == goal:
    return path, g
A* returns the g directly as it is tracking all along and not at the end like greedy.

Step 7: Keep exploring neighbours
for n, w in graph[node]:
    queue.append((g+w+h[n], g+w, path+[n]))
we use 'w' instead of discarding it.
g + w + h(n) calculates the new f
g + w(n) calculates new g

Step 8: Call & Print same as Greedy BFS



Consider S (start node)

$f(S) = 0 + 4 = 4$

queue contains (4, 0, [S])

S has 2 neighbours

S → A

$f(A) = 2 + 2 = 4$

queue has (4, 2, ['S', 'A'])

S → B

$f(B) = 3 + 1 = 4$

queue has (4, 1, ['S', 'B'])

(4, 2, ['S', 'A'])

(4, 1, ['S', 'B'])

Both have f has 4 so go with least 'g' which is B.

Exploring B:

B has one neighbour

B → G

$f(G) = 5 + 0 = 5$

add (5, 5, ['S', 'B', 'G'])

(4, 2, ['S', 'A'])

(5, 5, ['S', 'B', 'G'])

explore A:

A → G $f(G) = 12 + 0 = 12$

queue has

(5, 5, ['S', 'B', 'G'])

[12, 12, ['S', 'A', 'G']]

So the goal has been reached & the path is S → B → G

Cost is 5